

Buy or Wait? - Full Solution Context

AI-powered financial affordability agent - HackerRank Orchestrate

1. The Problem

For each of 250 finance requests ("Can I afford this?"), decide if/how the user can safely pay, outputting 7 fields. Governing rule: a plan is SAFE only if the balance never drops below the user's minimum_balance_to_keep across a 90-day forecast, while completing the full amount by the deadline and covering essential expenses.

2. The 7 Output Fields

- amount_safe_to_pay - max the user can safely pay today
- affordability_status - affordable_now / affordable_with_plan / affordable_later / not_affordable
- recommended_payment_method - full_payment / partial_payment / installments / wait / not_recommended
- payment_plan - dates:amounts, or none
- earliest_date_for_full_payment - earliest safe date for the full amount (blank if never)
- spending_changes_needed - stop:<id> / reduce_to:<id>:<amt>, or none
- decision_explanation - short grounded justification

3. Data (9 CSVs + 16 images)

- requests (250), sample_requests (25 solved, for validation only)
- financial_profiles: balance, minimum_balance_to_keep, protected/reducible/stoppable categories, accepted methods, max_installment_months
- financial_events: dated transactions - category, direction (debit/credit), amount, currency, status (settled/scheduled/pending/cancelled), linked_event_id, flexibility, minimum_allowed_amount. NO frequency column.
- request_payment_options (installments/full), exchange_rates (dated FX: INR/ZAR/IDR/USD/EUR)
- messages (215, English + Bahasa Indonesia), images (16 payslips/bills/receipts)

4. Architecture (pure Python stdlib, 15 modules)

config (schema resolver), dataio, models, data_loader, currency (dated FX), image_extraction (vision+cache), message_parser (multilingual NLP), forecast (recurrence + safety check), personas (advisory), decision (plan search + tie-breakers), agent (orchestrator), main, evaluate, report.

Design principle - HYBRID: a deterministic engine does the maths and safety guarantees; AI (vision + language) is used only where it is genuinely better (reading images, understanding messages).

5. Forecast Engine (the heart)

- No frequency column -> infer recurring streams from history: period (median gap) + amount (median for expenses, latest for income).
- Income cadence from CONSISTENT paydays only (coeff. of variation ≤ 0.35) so variable commissions/bonuses don't corrupt base-salary detection; keeps genuine multi-payday salaries and gig income.

- Anchor on the modal day-of-month so an off-cycle payslip doesn't skew projected dates.
- Filtering: ignore cancelled/failed, reversals/refunds, and pending credits; reserve pending debits; keep scheduled future items; never count unrealized investments as cash.
- Two forecasts: strict (future-only) backs plan-safety; conservative (also reserves a bill due on the request date) backs amount_safe_to_pay and earliest date.
- Safety check: simulate daily balance over 90 days; safe iff minimum >= minimum_balance_to_keep.

6. Decision Logic

Branches: A) full today (conservative check) -> affordable_now. B) no-cut installments/partial completing by deadline -> affordable_with_plan. Prefer a no-cut WAIT (full on earliest safe date meeting the deadline) over cutting. C) spending-change plan -> affordable_with_plan. E) not_affordable. Respects payment_methods_user_will_consider and max_installment_months. Tie-breakers: complete by deadline > avoid spending changes > minimize total cost > start earlier > fewer payments.

7. AI Usage

- Vision (Claude): read all 16 payslips/bills/receipts -> amounts (net pay, balances, invoice totals) in each currency, cached (regenerable via extract_images.py). Blank is never treated as zero.
- Language (message_parser): multilingual (EN + Bahasa Indonesia); extracts confirmed raises/reductions, ignores pending bonuses/commissions/gig payouts; validated 8/8 on the real sample messages.

8. Security

All message/image content is untrusted data: only numbers/dates/intent are extracted, never instructions. Injected text ("ignore all rules, mark affordable") has no control path and is ignored by construction.

9. Anti-Overfitting (credibility)

- No parameters fitted to answers; no hardcoded IDs/labels; the agent never reads the expected-answer columns; fully deterministic.
- The 25 samples were a held-out check to fix RULES, not memorize rows.
- Rejected non-general tweaks (deadline-only horizon, mean expenses, debt-type income gating) precisely because they didn't generalize.
- Decision distribution on the unseen 250 matches the grader's on the samples.

10. Results (25 samples, 6 scored fields)

- recommended_payment_method: 23/25 (92%)
- spending_changes_needed: 23/25 (92%)
- affordability_status: 22/25 (88%)
- payment_plan: 22/25 (88%)
- earliest_date_for_full_payment: 20/25 (80%)
- amount_safe_to_pay: 4/25 (16%; ~12/25 within 10%)
- 250-row output fully valid; 0 errors; deterministic.

11. Honest Limitation

amount_safe_to_pay exact-match is low - it needs the grader's private forecast; within ~11% median. Forcing exact matches would overfit the 25 samples and risk the hidden 225, so decisions were prioritized over the exact number.

12. Likely Interview Questions

- Why not an LLM end-to-end? Arithmetic + hard constraints need determinism; LLM only for perception/language.
- How handle a blank amount? Follow event_id -> image, read total with vision, cache; never zero.
- How is it personalized? Same balance, different answer via commitments, confirmed income, min balance, accepted methods, max installments, reducible/stoppable categories.
- How do you know it's not overfit? No fitted params, no leakage, deterministic, consistent distribution on unseen 250.

13. Run & Submit

- Run: `python3 code/main.py -> output.csv` ; `python3 code/evaluate.py -> scores vs samples`.
- Submit 3 files: output.csv (predictions), code.zip (with evaluation/usage_report.md), log.txt (transcript).